Module 3 — Relational Databases & SQL for Analytics

Every dataset you've worked with so far has lived in a CSV file or inline data. That works for small, single-user analysis — but it breaks the moment your data grows, your team grows, or your questions span multiple tables. Relational databases solve this. They enforce structure, guarantee integrity, and let you ask complex questions across connected data with a single SQL query.

This module takes you from flat files to structured databases. You'll design schemas, write analytical SQL, connect from Python, and build ETL pipelines that move data between PostgreSQL and Pandas.

What success looks like: By the end of this module, you can design a normalized relational schema for a business domain, write SQL queries (including JOINs, CTEs, and window functions) to answer analytical questions, connect to PostgreSQL from Python using psycopg3 and SQLAlchemy, and build a tested ETL pipeline that extracts, transforms, validates, and loads data.

In This Reading

1. Why Relational Databases?
2. Data Modeling Fundamentals
3. SQL Fundamentals
4. JOINs
5. Subqueries and Common Table Expressions
6. Window Functions
7. Working with PostgreSQL Locally
8. Python–Database Connectivity
9. ETL Patterns
10. Data Quality Validation

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Why Relational Databases?

You've been loading data with `pd.read_csv()` and saving results with `df.to_csv()`. CSVs are simple and portable — but they guarantee nothing about your data:

- **No type enforcement.** A "date" column can contain integers, strings, or blanks. Nothing stops it.
- **No relationships.** If your sales data references a customer, there's no guarantee that customer exists anywhere.
- **No concurrent access.** Two people editing the same CSV will overwrite each other's changes.
- **No transactions.** If your script fails halfway through writing, you get a corrupted file.

A relational database solves all of these problems. When you define a table, you declare what types each column holds, which values are required, and how tables relate to each other. The database **enforces** these rules — it rejects data that violates them.

PostgreSQL is the relational database you'll use in this course. It's open-source, production-grade, and used by companies from startups to large enterprises. You already pulled the `postgres:15-alpine` Docker image during Pre-Work and verified connectivity during EID 1.

SQLite is an embedded database that ships with Python (no server required). It's useful for prototyping, local development, and lightweight applications. You used it in Pre-Work Day 8. In this module, the drill uses SQLite for convenience; the lab and integration use PostgreSQL.

When to use each:

Scenario	PostgreSQL	SQLite
Multi-user production system	Yes	No
Concurrent write access	Yes	Limited
Local prototyping / embedded	Possible but heavy	Yes
CI/CD testing (lightweight)	Needs Docker service	Built-in
This course's lab + integration	Yes	—
This course's drill (Day 1 evening)	—	Yes

---

2. Data Modeling Fundamentals

Tables, Rows, and Columns

A relational database organizes data into **tables**. Each table represents an entity (employees, departments, projects). Each **row** is one instance of that entity. Each **column** is an attribute with a defined data type.

```sql
CREATE TABLE departments (
    dept_id   SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    location  VARCHAR(100),
    budget    NUMERIC(12, 2)
);
```

This DDL (Data Definition Language) statement creates a table with four columns. `SERIAL` auto-generates an integer ID. `PRIMARY KEY` enforces uniqueness. `NOT NULL` means the column cannot be empty. `NUMERIC(12, 2)` stores decimal numbers with 2 decimal places.

Primary Keys

Every table needs a **primary key (PK)** — a column (or set of columns) that uniquely identifies each row. Two types:

- **Natural key:** A real-world attribute that's already unique (e.g., email address, ISBN). Convenient but can change.
- **Surrogate key:** An artificial identifier (auto-incrementing integer or UUID) with no business meaning. Stable and predictable.

This course uses surrogate keys (`SERIAL` or `INTEGER PRIMARY KEY`) for simplicity and stability.

Foreign Keys

A **foreign key (FK)** is a column that references another table's primary key. It enforces **referential integrity** — every value in the FK column must exist in the referenced table.

```sql
CREATE TABLE employees (
    emp_id    SERIAL PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    dept_id   INTEGER REFERENCES departments(dept_id),
    hire_date DATE NOT NULL,
    salary    NUMERIC(10, 2) NOT NULL,
    title     VARCHAR(100)
);
```

Here, `dept_id` in `employees` references `dept_id` in `departments`. If you try to insert an employee with `dept_id = 99` and no department 99 exists, PostgreSQL rejects the insert.

Relationship Types

- **One-to-many:** One department has many employees. The FK lives on the "many" side (`employees.dept_id` → `departments.dept_id`).
- **One-to-one:** One employee has one parking spot. The FK lives on either side, with a UNIQUE constraint.
- **Many-to-many:** One employee can work on many projects; one project can have many employees. This requires a **bridge table** (also called a junction table):

```sql
CREATE TABLE project_assignments (
    assignment_id  SERIAL PRIMARY KEY,
    emp_id         INTEGER REFERENCES employees(emp_id),
    project_id     INTEGER REFERENCES projects(project_id),
    hours_allocated NUMERIC(6, 1),
    role           VARCHAR(50)
);
```

The bridge table holds pairs of foreign keys. Each row represents one employee-project relationship.

Normalization

Normalization is the process of organizing a schema to minimize redundancy. Consider a flat spreadsheet where every row repeats the department name and location for each employee. If the department moves to a new building, you update dozens of rows — and if you miss one, your data is inconsistent.

Normalization eliminates this by splitting data into related tables:

- **1NF (First Normal Form):** Every cell contains a single, atomic value. No repeating groups or arrays in a column.
- **2NF (Second Normal Form):** Every non-key column depends on the entire primary key (no partial dependencies). Relevant when using composite keys.
- **3NF (Third Normal Form):** Every non-key column depends directly on the primary key — not on another non-key column (no transitive dependencies). Example: if `employees` stores `dept_name` alongside `dept_id` , `dept_name` depends on `dept_id` (not on `emp_id`). Move it to the `departments` table.

Pragmatic Denormalization

Normalization is a design principle, not a religion. In analytics and reporting contexts, **denormalized** tables — with intentional redundancy — can dramatically simplify queries and improve read performance.

Common pattern: maintain a normalized transactional schema for data integrity, and create denormalized **reporting tables** or **materialized views** for analytics queries. The ETL pipeline you build in this module does exactly this: extract from normalized tables, transform (join, aggregate), and load into a summary table optimized for analysis.

---

3. SQL Fundamentals

SQL (Structured Query Language) is **declarative**: you describe *what* you want, and the database figures out *how* to get it. This is fundamentally different from Python, where you specify the exact steps.

SELECT, FROM, WHERE, ORDER BY

```sql
-- All employees in the Engineering department, sorted by salary
SELECT name, salary, title
FROM employees
WHERE dept_id = (SELECT dept_id FROM departments WHERE name = 'Engineering')
ORDER BY salary DESC;
```

- `SELECT` — which columns to return
- `FROM` — which table to query
- `WHERE` — which rows to include (filter before grouping)
- `ORDER BY` — how to sort the result (`ASC` default, `DESC` for descending)
- `LIMIT` — restrict the number of rows returned

Comparison and Filtering

```sql
WHERE salary > 50000                        -- comparison
WHERE title LIKE '%Engineer%'               -- pattern matching (% = wildcard)
WHERE dept_id IN (1, 3, 5)                 -- set membership
WHERE hire_date BETWEEN '2023-01-01' AND '2024-12-31'  -- range
WHERE location IS NULL                      -- null check (not = NULL)
WHERE location IS NOT NULL                  -- non-null check
```

`NULL` is not a value — it's the absence of a value. You cannot use `= NULL` or `!= NULL`. Always use `IS NULL` / `IS NOT NULL`.

Aliases

```sql
SELECT e.name AS employee_name, d.name AS department_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

`AS` creates an alias. Table aliases (`e` , `d`) shorten multi-table queries. Column aliases (`employee_name`) make output readable.

GROUP BY and Aggregate Functions

```sql
-- Total salary by department
SELECT d.name, COUNT(*) AS employee_count, SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.name;
```

Aggregate functions: `COUNT(*)`, `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`.

Rule: Every column in `SELECT` that is not inside an aggregate function must appear in `GROUP BY`. PostgreSQL enforces this strictly.

HAVING

```sql
-- Departments with total salary over 150,000
SELECT d.name, SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.name
HAVING SUM(e.salary) > 150000;
```

`HAVING` filters groups *after* aggregation. `WHERE` filters rows *before* aggregation. You cannot use aggregate functions in `WHERE`.

Query Execution Order

SQL's **syntax order** (how you write it) differs from its **execution order** (how the database processes it):

Step	Clause	What it does
1	`FROM` + `JOIN`	Identify source tables, combine rows
2	`WHERE`	Filter individual rows
3	`GROUP BY`	Group remaining rows
4	`HAVING`	Filter groups
5	`SELECT`	Choose columns, compute expressions
6	`ORDER BY`	Sort results
7	`LIMIT`	Restrict row count

This explains why you can't use a column alias from `SELECT` in `WHERE` — `WHERE` executes before `SELECT`. It also explains why `HAVING` can use aggregates but `WHERE` can't — aggregates don't exist until after `GROUP BY`.

---

4. JOINs

JOINs combine rows from two or more tables based on a related column (usually a foreign key).

INNER JOIN

Returns only rows where both tables have a matching value.

```sql
SELECT e.name, d.name AS department
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

If an employee has `dept_id = NULL` or references a department that doesn't exist (which FK constraints prevent), they're excluded.

LEFT JOIN

Returns all rows from the left table, plus matching rows from the right table. Where there's no match, right-side columns are `NULL`.

```sql
-- All employees, including those not assigned to any project
SELECT e.name, p.name AS project_name
FROM employees e
LEFT JOIN project_assignments pa ON e.emp_id = pa.emp_id
LEFT JOIN projects p ON pa.project_id = p.project_id;
```

Employees with no project assignments appear in the result with `NULL` for `project_name`. This is how you find unmatched rows.

RIGHT JOIN

The mirror of LEFT JOIN — all rows from the right table, plus matches from the left. Less common in practice; you can usually rewrite as a LEFT JOIN by swapping table order.

FULL OUTER JOIN

Returns all rows from both tables. Unmatched rows on either side get `NULL` for the other table's columns.

Self-Join

Joining a table to itself. Useful for hierarchical data.

```sql
-- Employees and their managers (assuming a manager_id column)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Multi-Table JOINs

Chain JOINs to traverse relationships:

```sql
-- Employee names with their project names (3-table join)
SELECT e.name, p.name AS project
FROM employees e
JOIN project_assignments pa ON e.emp_id = pa.emp_id
JOIN projects p ON pa.project_id = p.project_id;
```

Common Mistake: Cartesian Product

If you forget the `ON` condition, every row in one table pairs with every row in the other. With 60 employees and 15 projects, that's 900 rows instead of 80. Always verify your JOIN conditions.

---

5. Subqueries and Common Table Expressions

Subqueries

A subquery is a `SELECT` inside another SQL statement.

In WHERE clause (scalar):

```sql
-- Employees earning more than the company average
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

In WHERE clause (set — IN):

```sql
-- Employees in departments located in Amman
SELECT name
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Amman');
```

In FROM clause (derived table):

```sql
-- Average salary by department, then filter
SELECT dept_name, avg_salary
FROM (
    SELECT d.name AS dept_name, AVG(e.salary) AS avg_salary
    FROM employees e JOIN departments d ON e.dept_id = d.dept_id
    GROUP BY d.name
) AS dept_stats
WHERE avg_salary > 60000;
```

Common Table Expressions (CTEs)

CTEs use `WITH ... AS (...)` to define named, reusable query blocks. They make complex queries readable by breaking them into logical steps.

```sql
-- Departments where average salary exceeds company-wide average
WITH company_avg AS (
    SELECT AVG(salary) AS avg_salary
    FROM employees
),
dept_averages AS (
    SELECT d.name AS dept_name, AVG(e.salary) AS dept_avg
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    GROUP BY d.name
)
SELECT da.dept_name, da.dept_avg, ca.avg_salary AS company_avg
FROM dept_averages da, company_avg ca
WHERE da.dept_avg > ca.avg_salary;
```

When to use subqueries vs. CTEs:

- **Subqueries:** simple, one-off filters (e.g., `WHERE salary > (SELECT AVG...)`).
- **CTEs:** multi-step logic, when a result is referenced multiple times, or when readability matters (almost always in analytics).

Recursive CTEs can traverse hierarchical data (org charts, category trees). They're mentioned here for awareness — not required in this module's assignments.

---

6. Window Functions

Window functions perform calculations across a set of rows **related to the current row** — without collapsing the result set the way `GROUP BY` does. Every row retains its identity.

The OVER() Clause

```sql
SELECT name, dept_id, salary,
       AVG(salary) OVER(PARTITION BY dept_id) AS dept_avg_salary
FROM employees;
```

- `OVER()` defines the "window" — the set of rows the function looks at.
- `PARTITION BY` groups rows into partitions (like `GROUP BY` , but without collapsing).
- `ORDER BY` within `OVER()` defines the row order within each partition.

Without `PARTITION BY` , the window is the entire result set. Without `ORDER BY` , the function sees all rows in the partition at once.

Ranking Functions

```sql
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_de
FROM employees;
```

Function	Ties	Gaps
`ROW_NUMBER()`	Assigns unique sequential numbers (1, 2, 3) — breaks ties arbitrarily	No gaps
`RANK()`	Same rank for ties (1, 1, 3)	Gaps after ties
`DENSE_RANK()`	Same rank for ties (1, 1, 2)	No gaps

Use `ROW_NUMBER()` when you want exactly one winner per group (e.g., highest-salary employee per department). Use `RANK()` or `DENSE_RANK()` when ties matter.

Running Totals and Moving Averages

```sql
-- Running total of salaries within each department, ordered by hire date
SELECT name, dept_id, hire_date, salary,
       SUM(salary) OVER(PARTITION BY dept_id ORDER BY hire_date) AS running_total
FROM employees;
```

When `ORDER BY` is specified in `OVER()` , aggregate window functions default to a **frame** of all rows from the start of the partition through the current row — giving you a running total.

LAG and LEAD

Access the value from a previous or next row:

```sql
-- Compare each employee's salary to the previous hire's salary in the same dept
SELECT name, hire_date, salary,
       LAG(salary) OVER(PARTITION BY dept_id ORDER BY hire_date) AS prev_salary,
       salary - LAG(salary) OVER(PARTITION BY dept_id ORDER BY hire_date) AS sala
FROM employees;
```

`LAG(col, n)` looks back `n` rows (default 1). `LEAD(col, n)` looks forward.

Window Functions in WHERE

You **cannot** filter on a window function in the same query's `WHERE` clause — the window function hasn't been computed yet at the `WHERE` stage (see execution order). Wrap it in a CTE:

```sql
WITH ranked AS (
    SELECT name, dept_id, salary,
           ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM employees
)
SELECT name, dept_id, salary
FROM ranked
WHERE rn = 1;
```

---

7. Working with PostgreSQL Locally

You set up Postgres during EID 1. Here's a quick reference for the commands you'll use in the lab and integration.

Starting the Container

```bash
docker run -d \
  --name postgres-m3 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=module3 \
  -p 5432:5432 \
  -v pgdata_m3:/var/lib/postgresql/data \
  postgres:15-alpine
```

- `-d` runs in the background
- `--name` gives the container a name (stop it later with `docker stop postgres-m3`)
- `-e` sets environment variables (user, password, default database)
- `-p 5432:5432` maps the container's port to your machine
- `-v pgdata_m3:/var/lib/postgresql/data` persists data across container restarts

Connecting with psql

```bash
psql -h localhost -p 5432 -U postgres -d module3
```

You'll be prompted for the password (`postgres` in the example above).

Useful psql Commands

Command	What it does
`\dt`	List all tables
`\d tablename`	Describe a table (columns, types, constraints)
`\l`	List all databases
`\c dbname`	Connect to a different database
`\i filename.sql`	Execute a SQL file
`\q`	Quit psql

Creating a Database and Running Scripts

```bash
# Create a new database (from psql)
CREATE DATABASE levant_tech;

# Or from the command line
psql -h localhost -U postgres -c "CREATE DATABASE levant_tech;"

# Run schema and seed files
psql -h localhost -U postgres -d levant_tech -f schema.sql
psql -h localhost -U postgres -d levant_tech -f seed_data.sql
```

Port Conflicts

If port 5432 is already in use (another Postgres instance), map to a different port:

```bash
docker run -d ... -p 5433:5432 ... postgres:15-alpine
# Then connect with: psql -h localhost -p 5433 -U postgres
```

---

8. Python–Database Connectivity

Two libraries connect Python to PostgreSQL. Both were installed during EID 1.

psycopg3 — Raw SQL

psycopg3 (`psycopg`) is the low-level PostgreSQL adapter. You write SQL directly.

```python
import psycopg

# Connect
conn = psycopg.connect(
    host="localhost",
    port=5432,
    dbname="levant_tech",
    user="postgres",
    password="postgres"
)

# Execute a query
cursor = conn.cursor()
cursor.execute("SELECT name, salary FROM employees WHERE dept_id = %s", (1,))
rows = cursor.fetchall()  # list of tuples

for name, salary in rows:
    print(f"{name}: {salary}")

# Clean up
cursor.close()
conn.close()
```

Context manager (automatic cleanup):

```python
with psycopg.connect(host="localhost", dbname="levant_tech",
                      user="postgres", password="postgres") as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT name FROM departments")
        departments = cur.fetchall()
```

Parameterized queries — never use string concatenation:

```python
# CORRECT — parameterized (safe)
cursor.execute("SELECT * FROM employees WHERE name = %s", (user_input,))

# WRONG — string concatenation (SQL injection vulnerability!)
cursor.execute(f"SELECT * FROM employees WHERE name = '{user_input}'")
```

SQL injection is a critical security vulnerability. Parameterized queries ensure user input is treated as data, not as SQL code.

Writing data:

```python
cursor.execute(
    "INSERT INTO departments (name, location, budget) VALUES (%s, %s, %s)",
    ("Research", "Amman", 500000.00)
)
conn.commit()  # Required! Without commit, the insert is rolled back.
```

SQLAlchemy Core — Higher-Level Interface

SQLAlchemy provides a higher-level interface. For ETL and analytics, the key feature is seamless integration with Pandas.

```python
from sqlalchemy import create_engine

# Create engine (connection pool)
engine = create_engine("postgresql+psycopg://postgres:postgres@localhost:5432/lev

# Read SQL result directly into a DataFrame
import pandas as pd
df = pd.read_sql("SELECT * FROM employees", engine)

# Write a DataFrame to a table
df.to_sql("employee_summary", engine, if_exists="replace", index=False)
```

Connection string format: `postgresql+psycopg://user:password@host:port/dbname`

The `+psycopg` tells SQLAlchemy to use psycopg3 as the database driver.

`if_exists` options for `df.to_sql()`:

- `"fail"` — raise an error if the table exists (default)
- `"replace"` — drop the table and recreate it
- `"append"` — add rows to the existing table

When to Use Each

Use case	psycopg3	SQLAlchemy
Learning what's happening at the wire level	Yes	—
Raw SQL control, complex transactions	Yes	—
DataFrame integration (`pd.read_sql`)	—	Yes
ETL pipelines	—	Yes (convenience)
Cross-database portability	—	Yes

This module: Use psycopg3 in the lab (direct SQL practice). Use SQLAlchemy in the integration (ETL convenience). Both are used in practice — knowing when to reach for each is a professional skill.

SQLAlchemy ORM — Conceptual Overview

SQLAlchemy also has an **ORM (Object-Relational Mapping)** that maps Python classes to database tables. You'd define a `class Employee(Base)` and interact with rows as Python objects instead of writing SQL.

The ORM is powerful for application development but adds abstraction that can obscure what SQL is actually being generated. In this module, we use **SQLAlchemy Core** (engine + raw SQL strings) for transparency. The ORM is not required in any graded deliverable.

---

9. ETL Patterns

ETL stands for **Extract, Transform, Load** — the pattern for moving data between systems.

In Module 2, your pipeline was: CSV → Pandas → CSV. In Module 3, it becomes: **Database → Pandas → Database** (or file). The pipeline architecture is the same — modular functions, each testable independently.

The ETL Script Structure

```python
from sqlalchemy import create_engine
import pandas as pd

def extract(engine):
    """Pull data from source tables into DataFrames."""
    customers = pd.read_sql("SELECT * FROM customers", engine)
    orders = pd.read_sql("SELECT * FROM orders", engine)
    return {"customers": customers, "orders": orders}

def transform(data):
    """Join, clean, aggregate, derive."""
    merged = pd.merge(data["orders"], data["customers"], on="customer_id")
    # Filter, compute, aggregate...
    summary = merged.groupby("customer_id").agg(
        total_orders=("order_id", "count"),
        total_revenue=("line_total", "sum")
    ).reset_index()
    return summary

def validate(df):
    """Check data quality before loading."""
    checks = {}
    checks["no_null_ids"] = df["customer_id"].isna().sum() == 0
    checks["positive_revenue"] = (df["total_revenue"] > 0).all()
    checks["no_duplicates"] = df["customer_id"].is_unique

    for check, passed in checks.items():
        status = "PASS" if passed else "FAIL"
        print(f"  [{status}] {check}")

    if not all(checks.values()):
        raise ValueError("Data quality checks failed")
    return checks

def load(df, engine, csv_path):
    """Write results to database and file."""
    df.to_sql("customer_analytics", engine, if_exists="replace", index=False)
    df.to_csv(csv_path, index=False)
    print(f"Loaded {len(df)} rows to customer_analytics table and {csv_path}")

def main():
    engine = create_engine("postgresql+psycopg://postgres:postgres@localhost:5432

    print("Extracting...")
    data = extract(engine)
    print(f"  Extracted {sum(len(v) for v in data.values())} total rows")

    print("Transforming...")
    summary = transform(data)
    print(f"  Transformed to {len(summary)} customer summaries")

    print("Validating...")
    validate(summary)

    print("Loading...")
    load(summary, engine, "output/customer_analytics.csv")

    print("ETL complete.")

if __name__ == "__main__":
    main()
```

Key ETL Principles

Modularity: Each stage is a separate function. You can test `transform()` without a database connection by passing it test DataFrames.

Idempotency: Running the ETL twice produces the same result. Use `if_exists="replace"` (drop and recreate) or a truncate-and-reload pattern. Never append without deduplication.

Logging: Print statements at each stage show progress and row counts. In production, you'd use Python's `logging` module — for this course, `print()` is sufficient.

Error handling: If `validate()` raises, the pipeline stops before loading bad data. This is the ETL equivalent of "fail fast."

---

10. Data Quality Validation

Data quality checks are not optional. They are an engineering practice — like writing tests. Without them, bad data flows silently through your pipeline and produces wrong results.

Schema Checks

Verify the structure of your data:

```python
def validate_schema(df, expected_columns, expected_dtypes):
    """Check that DataFrame has expected columns and types."""
    for col in expected_columns:
        assert col in df.columns, f"Missing column: {col}"
    for col, dtype in expected_dtypes.items():
        assert df[col].dtype == dtype, f"{col}: expected {dtype}, got {df[col].dt
```

- Column presence: are all expected columns there?
- Data types: is `order_date` a datetime, not a string?
- Null constraints: are NOT NULL columns actually non-null?

Data Checks

Verify the content of your data:

```python
def validate_data(df):
    """Check data values and relationships."""
    checks = {}

    # Value ranges
    checks["positive_quantities"] = (df["quantity"] > 0).all()
    checks["valid_dates"] = df["order_date"].notna().all()

    # Uniqueness
    checks["unique_order_ids"] = df["order_id"].is_unique

    # Referential integrity (after join)
    checks["no_orphaned_orders"] = df["customer_name"].notna().all()

    # Row count sanity
    checks["reasonable_row_count"] = len(df) > 0

    return checks
```

When to Validate

ETL Stage	What to check
After extract	Schema (columns, types), row counts, basic null checks
After transform	Derived columns exist, aggregation results are reasonable, no unexpected nulls introduced by joins
Before load	All critical checks pass — this is the gate
After load	Confirm row count in target matches transformed output

---

Troubleshooting

`psql: connection refused` Postgres container not running. Check: `docker ps`. If the container isn't listed, start it: `docker start postgres-m3` (or re-run the `docker run` command).

`psycopg.OperationalError: connection refused` Same as above — or wrong port/host. Verify your connection parameters match the Docker container's port mapping.

`psycopg.OperationalError: FATAL: password authentication failed` Wrong password. Check the `POSTGRES_PASSWORD` environment variable you set when creating the container.

`psycopg.OperationalError: FATAL: database "..." does not exist` Create the database first: `psql -h localhost -U postgres -c "CREATE DATABASE dbname;"`

`pd.read_sql()` errors with psycopg connection Use a SQLAlchemy engine, not a raw psycopg connection: `engine = create_engine("postgresql+psycopg://...")`. Pandas expects an engine or a DBAPI2 connection string.

GROUP BY error: "column must appear in GROUP BY clause" Every non-aggregated column in SELECT must be in GROUP BY. PostgreSQL enforces this strictly.

Window function in WHERE clause You can't filter on a window function result in the same SELECT's WHERE clause. Wrap the query in a CTE, then filter in the outer query.

`df.to_sql()` type issues Pandas `object` dtype maps to `TEXT` in Postgres. For explicit control, pass `dtype={"col": sqlalchemy.types.Integer()}`.

Forgotten `conn.commit()` psycopg3 defaults to transaction mode. INSERT/UPDATE/DELETE require `conn.commit()`. Without it, the data is rolled back when the connection closes.

Docker volume: data lost after container restart Use a named volume (`-v pgdata_m3:/var/lib/postgresql/data`). Without it, stopping the container erases all data.

Key Terms

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — the four guarantees of a database transaction
aggregate function	A function that computes one value from many rows: COUNT, SUM, AVG, MIN, MAX
bridge table	A table that resolves a many-to-many relationship by holding pairs of foreign keys
Cartesian product	The result of a JOIN with no condition — every row paired with every other row
CTE	Common Table Expression — a named temporary result set defined with `WITH ... AS`
connection string	The URL specifying host, port, database, user, and password for a database connection
cursor	A database object for executing queries and fetching results (psycopg3 pattern)
data quality check	A programmatic test verifying that data meets schema and value constraints
denormalization	Intentionally adding redundancy to a schema to optimize read/query performance
engine	SQLAlchemy's connection pool and dialect manager
ETL	Extract, Transform, Load — the pattern for moving data between systems
foreign key	A column referencing another table's primary key, enforcing referential integrity
idempotent	An operation producing the same result whether run once or many times
JOIN	An SQL operation combining rows from two or more tables on a related column
normalization	Organizing a schema to minimize redundancy; measured in normal forms (1NF, 2NF, 3NF)
ORM	Object-Relational Mapping — mapping database tables to programming language classes
parameterized query	A query using placeholders (%s) instead of string interpolation, preventing SQL injection
primary key	A column (or columns) uniquely identifying each row in a table
referential integrity	The guarantee that foreign key values always point to existing primary key values
schema	The structural definition of a database: tables, columns, types, keys, constraints
SQL injection	A security vulnerability where unescaped user input alters SQL query behavior
subquery	A SELECT nested inside another SQL statement
surrogate key	An artificial primary key (auto-incrementing integer or UUID) with no business meaning
window function	A function computing a value across related rows using OVER(), without collapsing the result

---

Summary

- Relational databases enforce structure, types, and relationships that flat files cannot
- Normalization (1NF → 2NF → 3NF) reduces redundancy; denormalization is a pragmatic choice for analytics
- SQL is declarative — describe what you want, the database determines how
- JOINs combine tables on related columns; LEFT JOIN preserves unmatched rows
- CTEs make complex queries readable by naming intermediate results
- Window functions compute across related rows without collapsing the result set
- psycopg3 gives you raw SQL control; SQLAlchemy gives you DataFrame integration
- ETL (Extract → Transform → Validate → Load) is a testable, modular pipeline pattern
- Data quality validation is an engineering practice, not an afterthought

---

What's Next

Tonight (Day 1): The drill — three SQL functions against a SQLite database. Write them from memory.

Monday (Day 2): The lab — a full SQL analytics pack against a multi-table PostgreSQL database. Nine queries including schema design, plus a KPI brief.

Wednesday (Day 4): The integration — an ETL pipeline in Python. Extract from Postgres, transform with Pandas, validate, and load.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.